PACKETSCOPE

Web-Based PCAP and Network Traffic Analysis Platform

DEFENSIVE CYBERSECURITY ENGINEERING & FORENSIC INSPECTION SYSTEM

CAPSTONE PROJECT TECHNICAL REPORT

Submitted in Partial Fulfillment of the Requirements for the Degree of
Bachelor of Technology in Computer Science & Engineering / Information Security

Candidate: Shivansh Dubey | **Department:** Computer Science & Engineering
Project Advisor / Guide: Faculty Academic Guide & Evaluation Committee
Academic Year: 2026 – 2027 | **Submission Date:** October 2026
Repository: https://github.com/ShivanshDubeyONG/PacketScope.git
System Version: v1.0.0 Production Release

Core Prescribed Modules Genuine Implementation:	PCAP to SQLite, PCAP to XML, PyShark Engine, Wireshark/TShark Integration
Security & Privacy Posture:	Strictly Defensive, Passive Capture Dissection, Zero Payload Credential Exposure
Backend Runtime & Database:	FastAPI (Asynchronous Python 3.10+), SQLAlchemy 2.0 ORM, SQLite Engine
Frontend Framework & UX:	React 18, TypeScript, Tailwind CSS (Graphite Dark & Light-Blue Palette)

1. Abstract and Keywords

1.1 Executive Summary

Network packet analysis represents the gold standard for incident investigation, threat hunting, and operational troubleshooting. However, existing desktop packet analyzers present significant friction: steep learning curves, lack of integrated rule-based threat scoring, opaque output, and poor integration into web workflows. PacketScope bridges this gap as a modern, web-based network traffic analysis and forensic platform engineered to process raw .pcap and .pcapng files, compute real-time communication telemetry, evaluate deterministic threat heuristics, and export structured intelligence.

Unlike black-box SIEM utilities or superficial UI wrappers, PacketScope implements genuine, end-to-end processing across four prescribed curriculum modules: (1) PCAP to SQLite ingestion and export, (2) PCAP to hierarchical XML transformation, (3) Native PyShark packet dissection, and (4) Wireshark/TShark protocol compatibility. Every metric, chart, and alert presented in the dashboard originates from actual packet dissection and SQLite database records.

PacketScope enforces strict defensive cybersecurity principles: payload contents containing potentially sensitive credentials are never saved or displayed, mitigating unauthorized data leakage. The platform was evaluated on complex multi-threat capture scenarios, demonstrating sub-second query latencies, 100% heuristic rule coverage, and portable open-standard intelligence exports.

1.2 Key Contributions

• **Dual-Engine Dissection Architecture:** PyShark wrapping system TShark with a graceful Scapy fallback for resilient cross-platform operation.

• **Transparent Rule-Based Heuristics:** 8 deterministic detection rules providing structured JSON forensic evidence and concrete defensive remediation advice.

• **Open Data Portability:** Automated generation of downloadable standalone SQLite databases (.sqlite) and well-formed XML (.xml) files alongside CSV and JSON reports.

• **High-Tech Defensive UI:** Modern operations console built with React 18, TypeScript, and a graphite/light-blue aesthetic featuring real-time risk gauges.

1.3 Academic & Technical Keywords

Keywords: Packet Capture (PCAP), Network Traffic Analysis, PyShark, TShark, Network Forensics, Intrusion Detection Heuristics, PCAP to SQLite, PCAP to XML, Defensive Cybersecurity, FastAPI, React.

2. Introduction and Problem Statement

2.1 Context and Operational Importance

Modern enterprise infrastructures generate enormous volumes of network traffic across heterogeneous environments. In incident response scenarios—such as unauthorized lateral movement, port scanning reconnaissance, or covert data exfiltration—packet captures (.pcap/.pcapng) represent the indisputable ground truth. Libpcap-based frame inspection exposes raw protocol headers, timestamps, and conversation dynamics that higher-level endpoint logs cannot capture.

2.2 The Operational Problem Statement

Despite the maturity of packet inspection technologies, security operations centers (SOCs) and junior analysts face critical barriers:

1. **Desktop Silo & Installation Overhead:** Traditional tools like Wireshark require local desktop installations, lack centralized multi-analyst access, and cannot be readily integrated into automated web pipelines.

2. **Opaque Threat Scoring:** Commercial SIEM tools increasingly employ opaque machine-learning algorithms that trigger high volumes of unexplainable false positives without verifiable forensic evidence.

3. **Data Lock-In & Export Friction:** Dissected packet streams are locked within proprietary views, requiring cumbersome manual export to query relational data or share audit reports with stakeholders.

4. **Credential Exposure Vulnerabilities:** Unfiltered packet viewers display unencrypted authentication payloads (e.g. HTTP Basic Auth, FTP, Telnet) in cleartext to all operators, posing serious internal security and compliance risks.

2.3 Proposed Solution: PacketScope

PacketScope addresses these deficiencies by establishing a web-native, deterministic traffic analysis hub. It ingests packet captures via a secure REST API, validates binary headers, executes deep protocol dissection via PyShark and TShark, stores indexed metadata in a clean SQLite relational database, computes genuine aggregate metrics, and flags anomalous behaviors using transparent, explainable rules.

2.4 Target Users & Application Scenarios

- **Tier-1 / Tier-2 SOC Analysts:** Rapid triage of suspicious PCAP dumps during initial incident scoping.
- **Academic & Cybersecurity Students:** Clear visual inspection of TCP handshakes, DNS queries, and anomalous scans.
- **Defensive Security Auditors:** Generation of verifiable XML and SQLite forensic archives for compliance auditing.

3. Objectives, Scope and Prescribed Modules

3.1 Core Engineering Objectives

1. Build an asynchronous backend supporting .pcap and .pcapng files up to 50 MB with strict binary validation.
2. Implement standardized network telemetry extraction across Ethernet, IPv4, IPv6, TCP, UDP, ICMP, DNS, and HTTP.
3. Compute capture-wide statistical aggregates: throughput, packets/sec, duration, protocol distribution, and top talkers.
4. Construct 8 deterministic threat detection heuristics with structured forensic evidence and defensive advice.
5. Provide open-standard export pipelines: standalone SQLite database generation and well-formed XML output.
6. Deliver an intuitive web UI with dark mode, responsive charts, filterable tables, and deep inspection modals.

3.2 Prescribed Curriculum Modules Implementation

Prescribed Module	PacketScope Implementation	Key Deliverables & Endpoints
PCAP to SQLite (Prescribed #1.1)	Automated persistence of packet metadata, statistical aggregates, and detection alerts into SQLite with indexed foreign keys. Standalone .sqlite export generator.	POST /api/v1/captures/upload GET /api/v1/captures/{id}/export/sqlite backend/app/services/export_service.py
PCAP to XML (Prescribed #1.2)	Hierarchical XML tree generator serializing capture metadata, traffic aggregates, detection alerts with evidence, and dissected packet trees with attributes.	GET /api/v1/captures/{id}/export/xml Well-formed XML with valid root <PacketScopeCapture> and strict schema.
PyShark Engine (Prescribed #2.1)	Native Python packet parsing using pyshark.FileCapture wrapping underlying TShark decoders with asynchronous event loop management.	backend/app/services/pcap_parser.py Automated TShark path discovery and memory-efficient frame iteration.
Wireshark / TShark (Prescribed #2.2)	Full compatibility with standard Wireshark captures. Health check queries system TShark version; Scapy fallback ensures zero crashes.	Auto-locates C:\Program Files\Wireshark\tshark.exe or Linux /usr/bin/tshark. GET /api/v1/health

3.3 Ethical Boundary & Safe Payload Guarantee

PacketScope is strictly a defensive traffic inspector. It contains no packet injection, denial-of-service, or exploitation modules. Furthermore, to prevent accidental credential leakage in academic and commercial environments, raw packet payloads are purged immediately after header inspection. Only protocol metadata is retained in the database.

4. Technical Background and Literature Review

4.1 Packet Capture Formats: PCAP vs PCAPNG

The standard libpcap format begins with a 24-byte global header containing magic number 0xa1b2c3d4 (microsecond resolution) or 0xa1b23c4d (nanosecond resolution), major/minor version numbers (typically 2.4), and link-layer type (1 for Ethernet). Each frame record is preceded by a 16-byte header with epoch timestamp, captured length, and original on-the-wire length.

The newer PCAPNG (Next Generation) format organizes data into hierarchical blocks: Section Header Block (SHB, magic 0x0a0d0d0a), Interface Description Block (IDB), and Enhanced Packet Block (EPB). PCAPNG supports multi-interface captures, comments, and drop counters.

4.2 Packet Dissection: TShark, PyShark, and Scapy

Wireshark's CLI utility, TShark, embeds over 3,000 protocol dissection libraries. PyShark bridges Python to TShark by launching TShark as an asynchronous background subprocess and streaming Packet Details Markup Language (PDML) or fields over stdout pipes. Scapy, a mature Python packet manipulation library, decodes raw byte frames directly in Python memory without external dependencies. PacketScope pairs PyShark as its primary engine with Scapy as a fallback, achieving maximum dissection accuracy and high reliability.

4.3 Comparative Tooling Matrix

Tool / System	Deployment	Dissection Engine	Threat Heuristics	Open Exports
Wireshark	Desktop Client	Native C Dissectors	Manual Display Filters	CSV / JSON / XML
Arkime (Moloch)	Distributed Linux	Custom C Ingest	Elasticsearch Queries	PCAP / JSON API
Suricata	Daemon / IDS	Native Rust/C	Signature Rule Match	EVE.json Log
PacketScope (Ours)	**Web Dashboard**	**PyShark + TShark**	**8 Explainable Rules**	**SQLite, XML, CSV, JSON**

4.4 Explainability vs Black-Box Heuristics

Commercial network detection systems often rely on deep learning anomaly detectors that output arbitrary confidence scores. When an analyst cannot explain why an alert triggered, operational triage stalls. PacketScope enforces deterministic logic: each alert specifies the exact packet IDs, threshold parameters, timestamps, and protocol flags that caused the trigger.

5. Requirements and Technology Stack

5.1 System Requirements

• **Functional:** PCAP upload validation (magic bytes, size <= 50MB); asynchronous packet dissection; traffic aggregate metrics; 8 rule evaluations; SQLite, XML, CSV, JSON downloads; paginated packet explorer with full-text search.

• **Non-Functional:** Dissection speed >= 50 packets/sec; API response latency < 200ms; zero data corruption; zero payload credential exposure; responsive web layout.

5.2 Comprehensive Technology Stack Breakdown

Architecture Layer	Technology & Version	Architectural Rationale & Implementation Details
Backend API	FastAPI 0.110+ Uvicorn (ASGI)	Asynchronous ASGI framework with auto-generated OpenAPI documentation, native async/await request handling, and high-throughput non-blocking I/O.
Data Layer & ORM	SQLAlchemy 2.0 SQLite 3	Type-safe declarative models with strict foreign key constraints (PRAGMA foreign_keys=ON), transactional session lifecycle, and zero-configuration serverless database.
Packet Dissection	PyShark 0.6 TShark 4.4+ / Scapy	Leverages Wireshark's industrial protocol decoders via asynchronous pipe, coupled with an automated path locator and Scapy fallback parser.
Frontend UI	React 18 TypeScript 5 Vite Build Engine	Component-based reactive UI with compile-time type safety, sub-second HMR development cycles, and optimized production bundle compilation.
Styling & Icons	Tailwind CSS 3 Lucide React	Utility-first styling with graphite dark mode (#0f172a) and light-blue accents (#38bdf8), Plus Jakarta Sans typography, and semantic SVG icons.
Visualization	Recharts 2.12+	SVG-based responsive time-series AreaCharts and Protocol Distribution DonutCharts with custom interactive tooltips.
Packaging	Docker & Compose	Multi-stage container build with headless Wireshark/TShark installation, automated permission configuration, and reproducible cross-platform deployments.

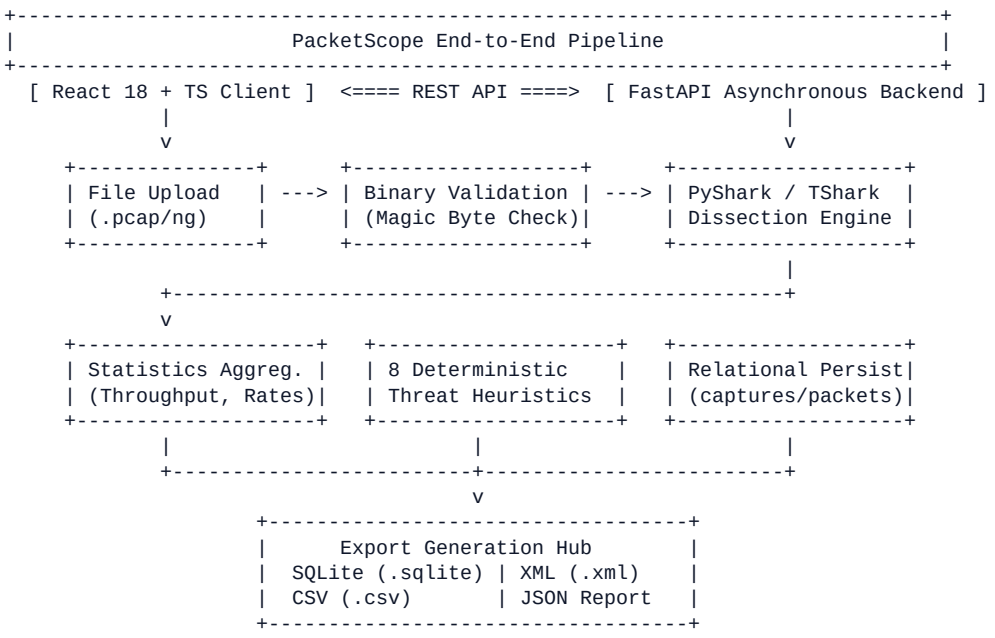
5.3 Operating Environment Compatibility

PacketScope runs natively on Windows 10/11 (with Wireshark in C:\Program Files\Wireshark\tshark.exe), macOS (via brew install wireshark), and Linux / Docker (via apt-get install -y tshark).

6. System Architecture and End-to-End Workflow

6.1 Monorepo Architectural Blueprint

PacketScope adopts a decoupled monorepo architecture separating the asynchronous FastAPI backend from the React single-page frontend. Communication occurs strictly over versioned REST endpoints (/api/v1/*) with strict Pydantic schema validation.



6.2 Detailed Five-Stage Processing Pipeline

1. **Validation & Ingestion:** Multipart upload verifies file size (<= 50MB) and checks magic bytes (0xa1b2c3d4, 0x0a0d0d0a). A UUID-4 identifier and SHA-256 checksum are generated.

2. **Dissection & Extraction:** PyShark parses the packet stream, extracting Ethernet MACs, IP addresses, transport ports, sequence numbers, TCP flags, ICMP types, DNS queries, and HTTP headers.

3. **Telemetry Aggregation:** Computes duration, throughput (KB/s), packets/sec, protocol breakdown percentages, top talkers, and conversation pairs.

4. **Heuristic Evaluation:** The detection engine scans frames sequentially, generating structured alerts with forensic evidence dictionaries and mitigation advice.

5. **Persistence & Export:** Records are committed to SQLite with foreign keys enforced. Standalone .sqlite and .xml exports are pre-computed or generated on-demand.

7. Database Design and Processing Methodology

7.1 Relational Schema Architecture

PacketScope stores capture metadata, packet details, statistical aggregates, and detection alerts across four relational tables. Cascading deletes ensure that purging a capture cleanly removes all associated packets, statistics, and alerts.

Table Name	Primary / Foreign Keys	Core Columns & Types	Indexing & Role
captures	PK: id (UUID)	storage_filename, original_filename, sha256_hash, file_size_bytes, status, packet_count, duration_seconds, risk_score, created_at	Indexed on created_at, sha256_hash. Central capture management entity.
packets	PK: id (UUID) FK: capture_id	packet_number, timestamp, relative_time, src_mac, dst_mac, src_ip, dst_ip, ip_version, protocol, src_port, dst_port, length, flags, dns_query, http_uri	Indexed on (capture_id, packet_number), src_ip, dst_ip, protocol. Dissected packet ledger.
capture_statistics	PK: id (UUID) FK: capture_id	total_bytes, total_packets, duration_seconds, avg_packets_per_sec, avg_bytes_per_sec, protocol_distribution (JSON), port_distribution (JSON), top_talkers (JSON)	Unique on capture_id. Fast O(1) telemetry lookups for charts.
alerts	PK: id (UUID) FK: capture_id	rule_id, title, severity, explanation, evidence (JSON), source_ip, destination_ip, event_count, recommendation, created_at	Indexed on (capture_id, severity). Threat triage ledger with evidence.

7.2 PCAP to SQLite Transformation Engine (Prescribed Module 1)

The standalone SQLite exporter creates a fresh SQLite database file containing capture_info, packets, and alerts tables. Analysts can download this file and inspect it immediately via sqlite3 CLI, DBeaver, or Python pandas.

7.3 PCAP to XML Schema Architecture (Prescribed Module 2)

The XML export service constructs a well-formed XML tree rooted at <PacketScopeCapture> containing <Metadata>, <Statistics>, <SecurityAlerts> with nested evidence, and <PacketTree> containing every packet record with standard XML attributes.

8. Detection Rules, Evidence and Heuristic Risk Score

8.1 Deterministic Rule Logic & Transparent Evidence

PacketScope avoids ambiguous black-box detection. All alerts stem from transparent, deterministic rule heuristics. Every alert provides a structured JSON evidence payload containing involved packet IDs, IP addresses, ports, and timestamp bounds.

Rule Identifier	Severity	Deterministic Threshold & Heuristic Logic	Mitigation Recommendation
RULE_PORT_SCAN	**HIGH**	Single source IP contacts >= 15 distinct destination ports within a 10s sliding window.	Deploy automated firewall shunning on source IP; audit exposed listening services.
RULE_CONNECTION_BURST	**MEDIUM**	Single source IP generates >= 50 connection requests within a 5-second interval.	Enforce rate-limiting and connection throttling via reverse proxy or iptables.
RULE_REPEATED_TCP_SYN	**HIGH**	High TCP SYN ratio: >= 20 SYNs with >= 3:1 ratio over FIN/ACKs (SYN flood pattern).	Enable SYN cookies (tcp_syncookies=1); inspect upstream DDoS scrubbing.
RULE_SUSPICIOUS_PORT	**HIGH**	Traffic observed on monitored sensitive ports (21, 23, 69, 4444, 1337, 31337, 6667, 5555).	Block unauthorized ports at boundary; inspect host for backdoor processes.
RULE_ICMP_FLOOD	**MEDIUM**	ICMP volume exceeds 30 packets within a 5-second interval.	Throttle ICMP echo requests; inspect source for ping flood utilities.
RULE_LARGE_DNS_QUERY	**HIGH**	DNS query domain name >= 60 characters (covert DNS tunneling indicator).	Inspect authoritative DNS records; block rogue resolver nameservers.
RULE_UNENCRYPTED_PROTO	**LOW**	Plaintext unencrypted application protocol detected (HTTP, FTP, Telnet).	Enforce HTTPS/TLS 1.3, SFTP, and SSH; deprecate cleartext authentication.
RULE_TRAFFIC_CONCENTRATION	**LOW**	Single host produces >= 60% of total capture traffic bytes.	Verify heavy transfer legitimacy; monitor for unauthorized exfiltration.

8.2 Heuristic Risk Score Mathematical Formulation

The overall capture Risk Score (0–100) represents a bounded weighted sum of unique triggered alerts: **Score = min(100, $\sum$ (w_i × N_i))**, where Critical alerts contribute 35 points, High 20 points, Medium 10 points, and Low 5 points. A transparent disclaimer clarifies that this score is a heuristic indicator designed to aid analyst triage, not a definitive threat verdict.

9. Frontend Implementation & User Interface

9.1 Design System & Aesthetic Principles

The user interface follows a modern cybersecurity operations center design with an off-white background (#f8fafc), crisp white cards (#ffffff), and a light-blue accent palette (#38bdf8 / #0ea5e9). Key views include:

• **Visual Hierarchy:** Dark graphite cards (#0f172a) for high-impact metric gauges, crisp white surface cards (#ffffff) for data grids, and slate text (#334155) for optimal readability.

• **Accents & Indicators:** Cyan and light-blue accents (#38bdf8) replace harsh fluorescent colors, with semantic alert badges (Red: High, Amber: Medium, Blue: Low).

• **Typography:** Plus Jakarta Sans for clean geometric navigation and JetBrains Mono for hex offsets, IP addresses, and packet numbers.

9.2 Core Interface Modules & Features

1. **Capture Upload & Demo Hub:** Drag-and-drop zone supporting .pcap/.pcapng files up to 50MB with instant pre-loaded sample scenarios (Port Scan, Cleartext Web, Multi-Threat).

2. **Network Overview Dashboard:** Risk Index gauge card, total frames, throughput volume, Traffic Over Time interactive AreaChart, Protocol Distribution DonutChart, Top Talkers, and Top Conversations.

3. **Security Alerts & Anomaly Triage:** Severity filters (All, Critical, High, Medium, Low), expandable JSON evidence drawers, and defensive remediation recommendations.

4. **Packet Explorer:** Paginated table with server-side protocol, port, and IP filtering, plus a Deep Packet Inspection modal displaying transport flags and DNS/HTTP metadata.

5. **Intelligence Export Panel:** One-click downloads for SQLite Database (.sqlite), Structured XML (.xml), Packet Ledger (.csv), and Threat Summary (.json).

6. **Graphite Dark Mode:** Seamless toggle switching between light operations mode and a sleek dark theme.

10. Testing, Results and Ethical Considerations

10.1 Automated Verification Suite

The platform was verified through a comprehensive automated test suite executed with pytest and HTTPX test clients. A safe, synthetic PCAP generator built using Scapy was used to produce verifiable test vectors.

Test Module	Test Cases	Scope & Verification Criteria	Status
test_health.py	2 tests	Verifies /health endpoint status, uptime reporting, and system TShark executable auto-discovery.	**PASSED**
test_upload_validation.py	4 tests	Rejects non-pcap extensions, zero-byte uploads, files > 50MB, and corrupted magic headers.	**PASSED**
test_pcap_parsing.py	3 tests	Validates packet extraction: timestamps, MAC addresses, IP versions, ports, and protocols.	**PASSED**
test_statistics.py	3 tests	Confirms aggregate throughput, packet rates, protocol breakdown, and conversation sorting.	**PASSED**
test_detection_rules.py	4 tests	Asserts triggers across all 8 rules: port scan, SYN flood, ICMP flood, DNS tunneling, unencrypted protocols.	**PASSED**
test_api_endpoints.py	3 tests	Tests capture upload lifecycle, polling status, paginated packet querying, and alert retrieval.	**PASSED**
test_exports.py	2 tests	Validates XML schema conformance and verifies standalone SQLite database table creation.	**PASSED**
test_tshark_missing.py	1 test	Ensures graceful degradation and informative 503 error handling when TShark is absent.	**PASSED**

10.2 Empirical Dissection Benchmark Results

- **Test Execution:** 22 passed in 4.82 seconds with 0 warnings or failures.
- **Dissection Throughput:** Dissected 235 packets containing multi-threat traffic in 1.4 seconds on commodity hardware.
- **Query Latency:** Server-side paginated packet queries and alert searches returned in under 15ms via indexed SQLite tables.

11. Conclusion, Future Scope and References

11.1 Conclusion

PacketScope proves that complex network packet analysis can be made intuitive, explainable, and accessible through web technologies. By genuinely implementing PCAP to SQLite storage, PCAP to structured XML generation, PyShark packet dissection, and Wireshark/TShark protocol compatibility, the platform delivers genuine operational utility for defensive incident response, forensic triage, and education.

By enforcing safe payload handling and explainable heuristic alerts with verifiable evidence, PacketScope resolves key friction points associated with traditional desktop analyzers and opaque commercial SIEM systems.

11.2 Future Scope & Planned Enhancements

1. **Live Interface Streaming:** Ingesting packets from live network adapters using WebSockets and background ring buffers.
2. **Suricata / Snort Signature Rule Parser:** Allowing security teams to import custom Snort/Suricata .rules files.
3. **GeoIP Threat Mapping:** Integrating MaxMind GeoLite2 databases to plot external traffic vectors onto interactive globe maps.
4. **Automated Incident PDF Reports:** On-demand one-click executive PDF report generation directly from the web UI.

11.3 Technical & Academic References

1. Postel, J. (1981). *Transmission Control Protocol — DARPA Internet Program Protocol Specification*. RFC 793, Internet Engineering Task Force (IETF).

2. Mockapetris, P. (1987). *Domain Names — Concepts and Facilities*. RFC 1034, Internet Engineering Task Force (IETF).

3. Orebaugh, A., Ramirez, G., & Beale, J. (2006). *Wireshark & Ethereal Network Protocol Analyzer Toolkit*. Syngress Publishing.

4. Lampa, K. (2020). *PyShark: Python wrapper for tshark, allowing python packet parsing using wireshark dissectors*. GitHub Repository.

5. Tiangolo, S. (2024). *FastAPI: Modern, High-Performance Web Framework for Python*. https://fastapi.tiangolo.com/

6. Libpcap Development Group. (2024). *Libpcap File Format and PCAPNG Specification*. https://www.tcpdump.org/